

NPCI/2025-26/IS/003

19th May 2025

To,

All Payment Service Providers (PSPs), Application Service Providers (ASPs) & Third-Party Application Providers (TPAPs) - Unified Payment Interface (UPI)

Subject: Comprehensive Mobile Application Security Framework for UPI- 2025

In continuation of our previous circular on Runtime Application Self-protection (Circular No: NPCI/2020-21/PSO/001 dated 28th April 2020), and Section 17 – “SDK Handling and Mobile Application Security” in “UPI Information Security Compliance Framework-2025” (Circular No: NPCI/2024-25/IS/006 dated 29th April 2024), we are issuing the detailed guidelines to incorporate mobile application security controls and enhancements in Run Time Application Self Protection (RASP) mechanisms for UPI application. Entities should submit the compliance against this circular from CERT-IN empanelled auditor once in a financial year along with compliance submission of “UPI Information Security Compliance Framework-2025” on or before December 31.

With the ongoing evolution of digital payments and the increasing sophistication of cyber threats, frauds and crimes, it is imperative to strengthen the security measures for mobile applications used for UPI transactions. The following updated controls are mandated to be implemented in all UPI applications:

A. Identify:**1. Root Detection and Root Cloaking Detection (Mandatory):**

- **Root Detection:** Implement root detection mechanisms that check for common indicators of a rooted device, such as the presence of root management apps, modified system files, and elevated permissions. UPI app must be disallowed from being installed on rooted / jail-broken mobile devices.
- **Root Cloaking Detection:** Use advanced techniques to detect root cloaking methods that attempt to hide the rooting status from security checks. This can include but not limited to monitoring for unusual system behaviours, detecting hooks in the UPI app's runtime environment and unexpected access to protected resources.

2. Application Installation Source Validation (Mandatory):

- UPI applications must validate the source of installation to ensure they are downloaded exclusively from authorized and verified distribution platforms

such as official app stores. Don't allow UPI application to run if downloaded from an unverified or unauthorized source.

- Implement measures to identify and remove rogue UPI applications from the Google Play Store, Apple App Store, and other online resources, including those on the dark web. Regularly monitor these platforms.

3. Harmful Application Detection (Recommended):

- Detect and alert users if blacklisted applications installed on mobile devices.

4. Virtual Device Detection (Recommended):

- Ensure UPI application should not be used on virtual devices/ emulators to prevent risk of automated attacks via bots, application tampering, etc.

5. Device Blacklisting (Recommended):

- Enable blocking of mobile devices from accessing UPI application, after certain malicious behaviour of the device such as root, jailbreak, dynamic code manipulation and hooking toolkits, proxy during runtime.
- Device blacklisting prevents unauthorized access and brute-force attacks, fortifying UPI application security and preserving user trust, ensuring seamless protection against malicious behaviour.

B. Protect:

1. Application Code and Data Integrity Protection at Runtime (Mandatory):

- **Memory Integrity Checks:** Implement continuous memory integrity checks to ensure that UPI application's code and data have not been altered during execution to verify the integrity of application code and data.
- **Runtime Verification:** Utilize runtime verification techniques to monitor UPI application's behaviour and detect any deviations from expected execution patterns. This can include but not limited to monitoring system calls, file accesses, and network communications.

2. Debugging Prevention (Mandatory):

- **Anti-Debugging Techniques:** Don't allow UPI application to run on mobile devices, detected with the presence of debugging tools (e.g., ptrace, gdb). The debugging tools can be detected by checking for debugger-related system flags and using timing checks to detect debugging delays.

3. Code Obfuscation and Encrypting Cryptographic Keys (Mandatory):

- **Code Obfuscation:** Apply code obfuscation techniques to prevent reverse engineering by incorporating methods including but not limited to renaming

variables and functions, inserting dummy code, and using control flow obfuscation.

- **Key Encryption:** Encrypt cryptographic keys using stringent encryption algorithm- AES 256 or equally stringent encryption algorithm and store them securely. Implement key management practices to ensure that keys are only accessible to authorized components.

4. Protection Against OWASP Top 10 Vulnerabilities (Mandatory):

- **Regular Updates and Patches:** Implement a proactive patch management process to regularly update the application and address the latest OWASP Top 10 vulnerabilities. This may include applying security patches, updating libraries, and fixing known issues.
- **Security Testing:** Conduct periodic security testing, with static and dynamic analysis (SAST and DAST), Secrets detection, and Software Composition Analysis to identify and mitigate vulnerabilities.

5. Hide App Screen During Navigation (Mandatory):

- **Screen Obfuscation:** Implement screen obfuscation techniques that hide sensitive information during navigation including blurring or masking the application screen when the user switches between apps or when the app is minimized.
- **Secure UI Components:** Use secure UI components that prevent unauthorized access to sensitive information. This can include implementing secure input fields and preventing screen capture.
- **Prevention of screenshot:** Prevent sensitive information from appearing in screenshots in UPI application by prohibiting the users from taking screenshots of the application to maintain data security and protect business interests.

6. Proxy Network Detection (Mandatory):

- **Network Analysis:** Perform network analysis to detect if UPI application is being accessed through a proxy network such as checking for unusual IP addresses, analysing network traffic patterns, and identifying proxy-related headers.
- **Access Control:** Implement access control mechanisms that restrict or block access from proxy networks including enforcing IP whitelisting, using geolocation checks, and requiring additional authentication.

7. VPN Network Detection (Recommended):

- Detect and prevent users from using UPI application when connected to VPN network, to prevent users from getting exposed to unauthorized access, and mitigating the risk of data interception and manipulation

8. SSL Pinning (Mandatory):

- Implement SSL pinning in UPI application with maximum of three public keys to prevent risk of Man-in-the-Middle (MitM) attacks.

9. Application Lock (Recommended):

- Protect UPI application with passcode, biometric (preferred) or pattern-based lock to prevent unauthorized access to sensitive data.

10. APK Locking (Recommended):

- Implement APK locking mechanism to prevent unauthorized redistribution and modification of UPI application, safeguarding against malicious distribution and maintaining app authenticity, ensuring robust protection against unauthorized access.

11. Application Permissions and Data Access (Recommended):

- **Application Permission:** Limit UPI application permissions to only those, which are necessary for functionality especially access to critical and sensitive UPI data.
- **Data Access Control:** Banks should ensure merchants/ UPI applications don't have visibility to data other than required or as may be required for reconciliation and customer service purposes mutually agreed by NPCI.

12. Developer Option and USB Debug Detection (Mandatory):

- **Developer Option Prevention:** Don't allow UPI application to run on mobile devices having developer option enabled, to prevent access to sensitive data by attackers or malicious applications.
- **USB Debug Detection:** Don't allow UPI application to run on mobile devices having USB debugging enabled, to prevent mobile devices from connecting to system with vulnerabilities.
- **Active USB Connection Detection:** Don't allow UPI application to run on mobile devices having active USB connection to prevent unauthorized data access, to safeguard against unauthorized data transfer and manipulation, ensuring robust protection against data breaches.

13. Wireless Debugging Prevention (Recommended):

- Don't allow UPI application to run on mobile devices having ADB (Android Debug Bridge) Wireless Debugging enabled. With ADB Wireless Debugging enabled, attackers can wirelessly connect the mobile device

and the machine on the same Wi-Fi to decode the flow and launch an attack subsequently.

14. Auto-Read OTP Implementation (Mandatory):

- **Permission Management:** Restrict OTP read permissions to necessary application components only.
- **Human Intervention:** Don't allow users to enter OTP manually.
- **Call based OTP:** Don't share OTP over calls. Leverage SMS based OTP mechanism only. [how this could prevent the attack and what led to this measure](#)

15. Sender ID Validation for Auto-Read OTP (Mandatory):

- **Whitelist Trusted Sender IDs:** Maintain a list of trusted sender IDs for OTP messages and verify the same before processing OTPs. Ensure OTPs sent via trusted banks only are processed.
- **Alerting Mechanism:** Generate alerts for OTPs sent from unrecognized sender IDs.

16. Registration Security on Older App Versions (Mandatory):

- **Force Update:** Prompt users to update to the latest app version for enhanced security.
- **Deprecate Old Versions:** Gradually phase out support for older app versions.
- **Security Patches:** Regularly release security patches for older versions until deprecated.

17. SMS Sent Check Validation during device binding (Mandatory):

- **Delivery Confirmation:** Implement mechanisms to confirm SMS delivery status.
- **Legitimate VMN:** Ensure SMSs are being sent to legitimate VMN (Virtual Mobile Number) of banks. [what are VMN](#)
- **Retry Logic:** Implement retry logic to ensure a reasonable number of attempts are made for undelivered SMS messages before marking them as failed.

18. Whitelist Private API for iOS Devices during device binding (Mandatory):

- Ensure Application ID is registered with NPCI to get the same whitelisted at Apple to access private API during user onboarding.
- **Disable editing in SMS body sent during device binding.**

19. Prevention of Dynamic Instrumentation and Runtime Manipulation (Recommended):

- Don't allow UPI application to run on mobile devices having Dynamic Instrumentation and Runtime Manipulation toolkit such as: Frida, Cycrypt, Ghidra etc. installed.
- Since Frida toolkit can be used to inject custom malicious scripts into black-box processes (running applications) on mobile device, preventing the same would prohibit in memory manipulation of application logic, thus ensuring integrity of UPI application.

20. Keylogger Prevention (Recommended):

- Don't allow UPI application to run on mobile devices having keyloggers installed to safeguard against unauthorized data capture, ensuring data confidentiality, and preventing theft of sensitive UPI information.

21. Screen Overlay (Recommended):

- Don't allow to display any application over UPI application to prevent Screen Overlay, to prevent unauthorized manipulation of user interfaces, safeguarding against tapjacking and preserving user privacy, ensuring robust protection against unauthorized actions.

22. Reverse Engineering Prevention (Mandatory):

- **Anti-Reverse Engineering Techniques:** Integrate anti-reverse engineering techniques that detect and prevent to attempts to reverse-engineer UPI application including but not limited to monitoring for the use of disassembly tools, detecting and preventing unusual execution patterns, and employing code obfuscation.
- **Behavioural Analysis:** Use behavioural analysis to identify and prevent suspicious activities that may indicate reverse engineering attempts. This can include monitoring for repeated failed authentication attempts, unusual API calls, and abnormal application behaviour.

23. Screen Mirroring Prevention (Mandatory):

- Don't allow UPI application to run on mobile devices having active screen mirroring in the background to prevent unauthorized monitoring of app interactions, safeguarding against data interception, and preserving user privacy, ensuring robust protection against malicious surveillance.

C. Detect:

1. Responding to Runtime Attacks (Mandatory):

- **Tamper Detection:** Implement tamper detection mechanisms that monitor UPI application's runtime environment for signs of tampering including but

not limited to checking for modifications to code, memory, & configuration files.

- **Customizable Actions:** Define customizable actions that the application can take in response to detected runtime attacks including restoring tampered code from a secure backup, degrading app performance to hinder the attack, or terminating the app to prevent further damage.

2. Unsecured Wi-Fi Detection (Recommended):

- Detect and alert users, when connected to open or unencrypted Wi-Fi networks, as it may lead to Man-in-the-Middle (MitM) attacks, causing loss of sensitive data and user privacy.

D. Respond:

1. Alerting Monitoring Systems (Mandatory):

- **Real-Time Alerts:** Configure UPI application to send real-time alerts to App providers' monitoring systems when a threat is detected with detailed information about the attack, such as the type of threat, affected components, and timestamp.
- **Automated Response:** Integrate automated response mechanisms that allow monitoring systems to take immediate action based on the alerts. This can include but not limited to reducing access rights, initiating countermeasures, or triggering security updates.

This Mobile Application Security Framework provides a comprehensive approach in securing UPI applications against a wide range of threats and vulnerabilities, ensuring the integrity and safety of UPI transactions.

By adhering to these guidelines in this framework and maintaining robust mobile application security practices, PSPs and TPAPs can significantly reduce the risk of potential security incidents and cyber frauds, mitigate the impact if an attack occurs, and recover effectively.

Yours sincerely,

Pravin Kumar

Chief Market Information Security Officer & DPO

9

MI